The first approach—injecting services into an endpoint handler—is the most common way to access the root of a dependency graph. You should use this approach in almost all cases in your minimal API applications. You can inject a service into an endpoint handler by adding it as a parameter to your endpoint handler method, as you saw in chapters 6 and 7 when you injected a `LinkGenerator` instance into your handler.

Listing 8.7 Injecting the `LinkGenerator` service in an endpoint handler

```
app.MapGet("/links", (LinkGenerator links) =>     ❶
{
    string link = links.GetPathByName("products");
    return $"View the product at {link}";
});
```

❶ The DI container creates a LinkGenerator instance and passes it as the argument to the handler.

The minimal API infrastructure sees that you need an instance of the `LinkGenerator`, which is a service registered in the container, and asks the DI container to provide an instance of the service. The DI container creates a new instance of `LinkGenerator` (or reuses an existing one) and returns it to the minimal API infrastructure. Then the `LinkGenerator` is passed as an argument to invoke the endpoint handler.

NOTE Whether the DI container creates a new instance or reuses an existing instance depends on the lifetime used to register the service. You'll learn about lifetimes in chapter 9.

As already mentioned, the DI container creates an entire dependency graph. The `LinkGenerator` implementation registered with the DI container declares the dependencies *it* requires by having parameters in its constructor, just as the `EmailSender` type from section 8.1 declared its